Bucles y Ciclos de Repetición 🔄

En un ejercicio donde tenemos que imprimir los números del 1 al 10, actualmente, nuestro código se vería de la siguiente manera:

```python
print("Número: 1")
print("Número: 2")
print("Número: 3")
print("Número: 4")
print("Número: 5")
print("Número: 6")
print("Número: 7")
print("Número: 8")
print("Número: 9")
print("Número: 10")
```

Copiar

Imaginemos que ahora tenemos que hacerlo para 1 millón de números, ejecutar todos esos `print()` sería demasiado repetitivo y definitivamente necesitamos algo más eficiente. Acá es donde los `ciclos`, `bucles` o `estructuras iterativas` entran en acción.

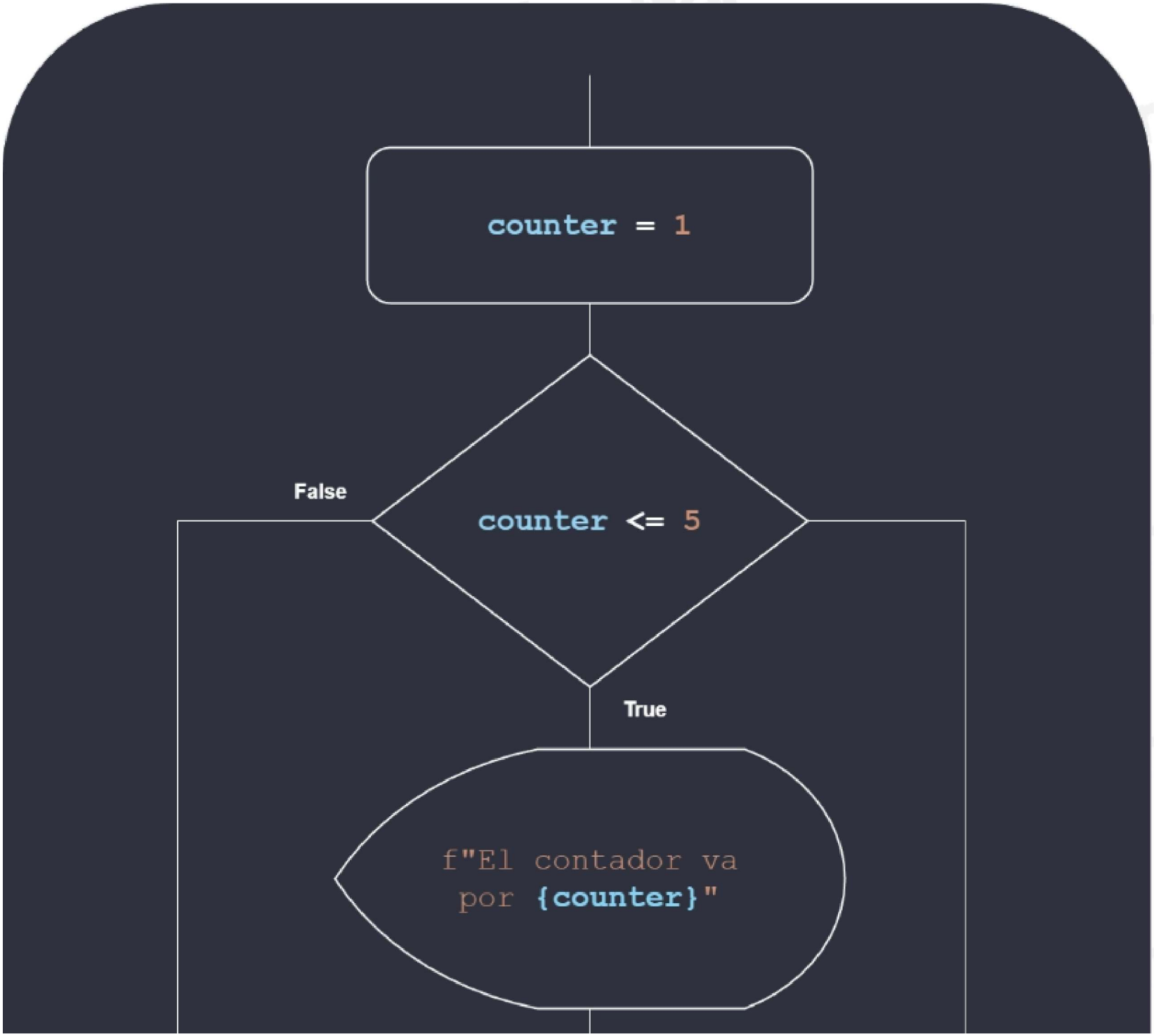
Estas estructuras nos permiten ejecutar un bloque de código repetidamente mientras se cumpla una condición o hasta que hayamos recorrido una colección de datos. En esta lección aprenderemos a implementarlas en `Python`.

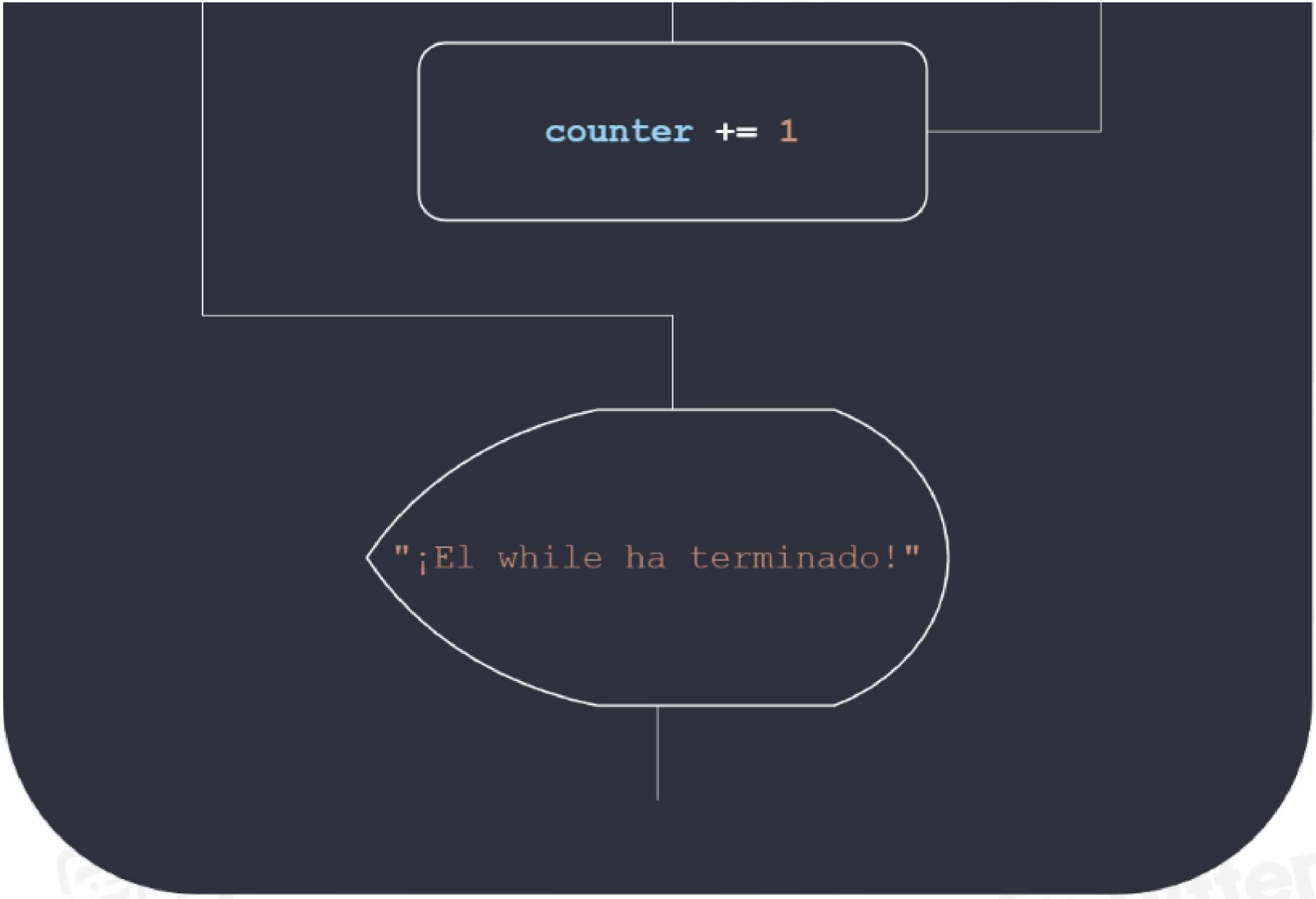
El ciclo `while`

El ciclo `while` (que significa "*mientras*") es la estructura que utilizamos cuando queremos ejecutar un bloque de instrucciones **mientras una condición sea verdadera (`True`)**.

Usamos `while` cuando **no sabemos** con exactitud cuántas veces se tiene que repetir una acción. El ciclo evaluará la condición antes de cada ejecución, si es `True`, entra al bloque, si es `False`, el ciclo termina y el programa continúa con la siguiente instrucción fuera del bucle.

Por ejemplo en este diagrama de flujo creamos un ciclo `while`, donde se va a imprimir el valor de la variable `counter` hasta que su valor sea mayor a 5.


counter = 1
counter <= 5
False
True
f"El contador va
por {counter}"



Ese mismo algoritmo lo podemos representar en `Python` de la siguiente manera:

```python
counter = 1

# El ciclo se ejecutará mientras contador sea menor a 5
while counter <= 5:
    print(f"El contador va por {counter}")

    # Incrementamos el contador para que la condición eventualmente sea False
    counter += 1

print("¡El while ha terminado!")
```

Copiar

```
El contador va por 1
El contador va por 2
El contador va por 3
El contador va por 4
El contador va por 5
¡El while ha terminado!
```

En este segundo ejemplo, vamos a repetir hasta que el usuario escriba específicamente la palabra "*salir*". No sabemos si lo va a hacer a la primera o en el intento 1000, la cantidad va a depender de cada usuario y siempre es incierto.

```python
favorite_word = input("Digite su palabra favorita: ")

# El ciclo continuará mientras lo que escribamos NO sea "salir"
while favorite_word != "salir":
    print(f"Su palabra favorita es '{favorite_word}'. ")
    favorite_word = input("Digite otra palabra favorita: ")

print("¡Sesión finalizada con éxito!")
```

Copiar

✓
```
Digite su palabra favorita: queso
Su palabra favorita es 'queso'.
Digite otra palabra favorita: jamon
Su palabra favorita es 'jamon'.
Digite otra palabra favorita: salir
¡Sesión finalizada con éxito!
```

Debemos tener precaución con ciertos casos, ya que si la condición nunca cambia a `False`, creamos un ciclo infinito. 🔁

```python
# ¡Cuidado! Esto se ejecutará para siempre si no lo detenemos manualmente.
while True:
    print("Esto es un ciclo infinito")
```

Copiar

💡 Cuando queremos detener un ciclo infinito o cualquier programa de `Python`, presionamos el comando `Ctrl + C` para detenerlo.

El ciclo `for`

El ciclo `for` está diseñado para recorrer elementos de una secuencia como una lista, un `string` o un rango de números. Utilizamos el `for` cuando tenemos certeza de cuántas veces queremos que se ejecute el ciclo o cuando necesitamos procesar una colección de datos paso a paso. A este proceso de recorrer elementos lo llamamos `iteración`.

Por ejemplo en este diagrama de flujo creamos un ciclo `for`, donde se va a imprimir cada elemento de `list_of_car_brands`.

El cual lo podemos aplicar el `Python` de la siguiente manera:

```python
list_of_car_brands = ["Mercedes Benz", "Toyota", "Mazda", "Hyundai"]

# Por cada elemento en la lista, ejecutamos el bloque
for car_brand in list_of_car_brands:
    print(f"Ejecutando ciclo para marca: {car_brand}")

print("¡Ciclo completado!")
```

Copiar

💡 **Dato importante:** La variable `car_brand` en el ejemplo anterior es creada automáticamente por `Python` y va tomando el valor de cada elemento de la lista en cada iteración.

La función `range()`

A veces queremos usar un `for` no para recorrer una lista existente, sino para repetir una acción un número específico de veces. Para esto, nos apoyamos en la función `range()`.

`range()` **genera** una secuencia de números. Es el compañero ideal del `for` cuando necesitamos, por ejemplo, ejecutar un código exactamente 10 veces sin tener una lista previa de elementos.

```python
# range(5) genera números del 0 al 4 (5 números en total)
for number in range(5):
    print(f"Número: {number}")
```

Copiar

✓
Número: 0
Número: 1
Número: 2
Número: 3
Número: 4

💡 El uso de la variable `i` es muy común a la hora de trabajar con posiciones, viene del inglés "*index*" y es parte de las reglas "*no escritas*" de los devs.

Esta función es muy versátil ya que podemos especificar desde qué número comenzar. El primer parámetro será el inicio (*incluido*) y el segundo el final (*excluido*).

```python
# Genera números del 5 al 9
for i in range(5, 10):
    print(f"Número: {i}")
```

Copiar

✓
Número: 5
Número: 6
Número: 7

```
Número: 8
Número: 9
```

También podemos indicar de cuánto en cuánto queremos avanzar. El tercer valor controla el incremento entre cada número de la secuencia.

```python
# Genera números pares del 0 al 10 (0, 2, 4, 6, 8, 10)
for i in range(0, 11, 2):
    print(f"Par: {i}")
```

Copiar

```
Par: 0
Par: 2
Par: 4
Par: 6
Par: 8
Par: 10
```

Control de ciclos 🚦

En ocasiones, necesitamos alterar el flujo normal de un `ciclo` mientras se está ejecutando. Para ello, podemos utilizar dos palabras clave: `break` y `continue`.

Comando `break`

Usamos `break` para forzar la finalización de un ciclo. En el momento en que se ejecuta un `break`, el bucle se detiene por completo, sin importar si la condición seguía siendo `True` o si quedaban elementos por recorrer.

```python
# Recorremos del 1 al 9
for number in range(1, 10):
    if number == 4:
        print("Número 4 encontrado, deteniendo ciclo...")
         # Salimos del for completamente
        break
    print(f"Revisando número: {number}")
```

Copiar

```
Revisando número: 1
Revisando número: 2
Revisando número: 3
Número 4 encontrado, deteniendo ciclo...
```

Comando `continue`

A diferencia del anterior, `continue` no detiene el ciclo por completo. Lo que hace es saltarse el resto del código en la iteración actual y saltar directamente a la siguiente. Vemos en el siguiente ejemplo que el número tres no se imprime como los demás, y salta al número 4.

```python
# Recorremos del 1 al 4
for number in range(1, 5):
    if number == 3:
        print("Saltando el número 3...")
        continue # Ignora lo que sigue y va a la siguiente vuelta
    print(f"Imprimiendo número: {number}")
```

Copiar

```
Imprimiendo número: 1
Imprimiendo número: 2
Saltando el número 3...
Imprimiendo número: 4
```
